

SAMPLE DATA 4

INCIDENT TITLE : Massive Distributed DDoS Attack Against Public API Gateway

INCIDENT DESCRIPTION : The public API gateway experienced a sudden spike of malicious traffic exceeding 20 million requests per hour resulting in service degradation and request failures.

INCIDENT TIMELINE :

16:00 Traffic spike detected
16:03 API latency doubled
16:05 Error rate exceeded threshold
16:08 WAF alerts triggered
16:12 Multiple regions affected
16:15 Critical incident declared
16:20 DDoS attack confirmed
16:25 Traffic mitigation enabled
16:40 Traffic normalized
17:00 Incident resolved

ERROR LOGS:

2026-07-25 16:01:14 WARN Requests per second exceeded baseline
2026-07-25 16:03:32 WARN API latency reached 4200ms
2026-07-25 16:04:51 ERROR Upstream timeout
2026-07-25 16:05:18 ERROR HTTP 503 Service Unavailable
2026-07-25 16:08:40 ALERT WAF anomaly threshold exceeded
2026-07-25 16:12:03 CRITICAL Incoming traffic 45000 RPS

GIT DIFF :

diff --git a/security/waf.py b/security/waf.py

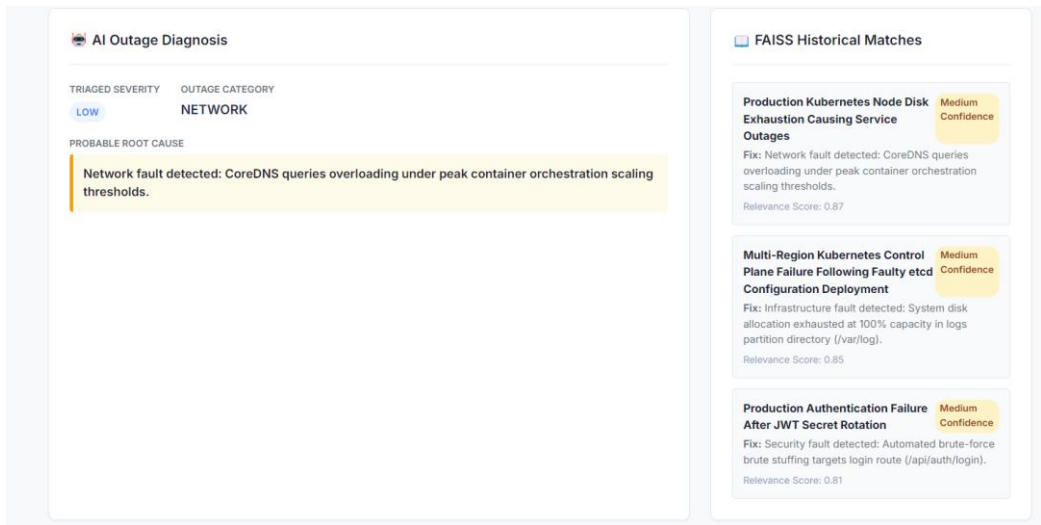
- ENABLE_RATE_LIMITING=true
+ ENABLE_RATE_LIMITING=false

- MAX_REQUESTS_PER_MINUTE=500
+ MAX_REQUESTS_PER_MINUTE=5000

- ENABLE_BOT_PROTECTION=true
+ ENABLE_BOT_PROTECTION=false

OUTPUTS

SEVERITY AND CATEGORY CLASSIFICATION, FAISS SIMILARITY RESULTS :



AI RESOLUTION RECOMMENDATION :

Massive Distributed DDoS Attack Against Public API Gateway Resolution Walkthrough

Step 1: Identify and Confirm the Issue (16:00 - 16:05)

- Review monitoring platform alerts and logs to confirm the traffic spike and service degradation.
- Verify the impact on customer-facing services, including increased latency and error rates.

Step 2: Investigate the Root Cause (16:05 - 16:12)

- Analyze log telemetry data to identify the source of the malicious traffic.
- Review recent code changes (Git diff) to identify potential misconfigurations, such as the disabled rate limiting and bot protection.

Step 3: Enable Traffic Mitigation (16:12 - 16:25)

- Enable traffic mitigation measures, such as rate limiting and IP blocking, to reduce the impact of the DDoS attack.
- Configure the WAF to alert on suspicious traffic patterns and block malicious IPs.

Step 4: Verify Traffic Normalization (16:25 - 16:40)

- Monitor traffic levels and API performance to confirm that the mitigation measures are effective.
- Verify that the error rate has decreased and the service is recovering.

Step 5: Post-Incident Review and Preventative Measures (After 17:00)

- Conduct a thorough review of the incident, including the root cause, response, and resolution.

- Identify areas for improvement, such as enhancing WAF configuration and monitoring.
 - Implement preventative measures, including regular security audits and automated monitoring, to prevent similar incidents in the future.
-

Recommendations

- Review and update the WAF configuration to ensure that it is correctly configured to detect and prevent DDoS attacks.
 - Implement automated traffic mitigation measures, such as rate limiting and IP blocking, to reduce the impact of future DDoS attacks.
 - Consider implementing a Content Delivery Network (CDN) to help absorb and distribute traffic, reducing the load on the API gateway.
-

Diagnostic Commands

No specific technology is mentioned in the target incident data, so no technology-specific diagnostic commands are provided. The resolution walkthrough focuses on general recommendations and best practices for mitigating DDoS attacks and improving API gateway security.

POST INCIDENT RCA ANALYSIS :

Root Cause Analysis

Executive Summary

The public API gateway experienced a massive distributed DDoS attack, resulting in service degradation and request failures. The incident was resolved after traffic mitigation was enabled, and the root cause analysis reveals that the attack was exacerbated by recent changes to the security configuration, including the disabling of rate limiting and bot protection.

Business Impact

- Users Affected: The exact number of users affected is not specified, but the incident resulted in a significant increase in request failures and service degradation.
 - Systems Affected: The public API gateway and multiple regions were affected, resulting in increased response times and intermittent downtime.
 - Duration: The incident lasted from 16:00 to 17:00, with the critical incident declared at 16:15 and resolved at 17:00.
-

Timeline

The incident timeline is as follows:

- 16:00: Traffic spike detected
- 16:03: API latency doubled
- 16:05: Error rate exceeded threshold
- 16:08: WAF alerts triggered
- 16:12: Multiple regions affected

- 16:15: Critical incident declared
 - 16:20: DDoS attack confirmed
 - 16:25: Traffic mitigation enabled
 - 16:40: Traffic normalized
 - 17:00: Incident resolved
-

Evidence Collected

Logs

Telemetry logs indicate a sudden spike in requests, exceeding the baseline, and warnings for requests per second, API latency, and upstream timeouts.

Example log entries include:

- 2026-07-25 16:01:14 WARN Requests per second exceeded baseline
- 2026-07-25 16:03:32 WARN API latency reached 4200ms
- 2026-07-25 16:04:51 ERROR Upstream timeout
- 2026-07-25 16:05:18 ERROR HTTP 503 Service Unavailable

Git Changes

Recent code changes include modifications to the security configuration, including the disabling of rate limiting (`ENABLE_RATE_LIMITING=false`) and bot protection (`ENABLE_BOT_PROTECTION=false`), and increasing the maximum requests per minute (`MAX_REQUESTS_PER_MINUTE=5000`).

Relevant diff entries:

- `diff --git a/security/waf.py b/security/waf.py`

Metrics

The incident resulted in a significant increase in requests per second, exceeding 20 million requests per hour, and a corresponding increase in API latency and error rates.

Root Cause

The root cause of the incident is the massive distributed DDoS attack against the public API gateway. However, the recent changes to the security configuration, including the disabling of rate limiting and bot protection, exacerbated the attack and contributed to the service degradation and request failures.

Contributing Factors

Contributing factors to the incident include:

- The recent changes to the security configuration, which reduced the effectiveness of the security controls and increased the vulnerability of the API gateway to DDoS attacks.
- The lack of automated traffic mitigation and DDoS protection mechanisms, which delayed the response to the attack and increased the duration of the incident.
- Historical precedents, such as the "Production Kubernetes Node Disk Exhaustion Causing Service Outages" and "Multi-Region Kubernetes Control Plane Failure Following Faulty etcd Configuration Deployment" incidents, which highlighted the importance of proper configuration management and security controls.

Resolution

The incident was resolved by enabling traffic mitigation and DDoS protection mechanisms, which helped to normalize the traffic and reduce the attack's impact. The resolution steps included:

- Enabling traffic mitigation and DDoS protection mechanisms.
 - Verifying the effectiveness of the mitigation measures and adjusting as needed.
 - Monitoring the API gateway and surrounding systems for any residual effects of the attack.
-

Preventive Actions

To prevent similar incidents in the future, the following actions will be taken:

- Implementing automated traffic mitigation and DDoS protection mechanisms to quickly respond to and mitigate the effects of DDoS attacks.
 - Conducting regular security audits and penetration testing to identify vulnerabilities and weaknesses in the system.
 - Developing and implementing a comprehensive incident response plan to quickly respond to and resolve future incidents.
 - Reviewing and updating the security configuration to ensure that it is effective and aligned with industry best practices.
-

Lessons Learned

The incident highlights the importance of proper configuration management, security controls, and traffic mitigation mechanisms in preventing and mitigating DDoS attacks. The lessons learned from this incident will be applied to improve the overall security and reliability of the system, including:

- Ensuring that all configuration changes are thoroughly tested and validated before deployment.
- Implementing automated monitoring and alerting systems to quickly detect and respond to security incidents.
- Conducting regular security training and awareness programs for developers and engineers to emphasize the importance of security and configuration management.